

CIS6003 - Advanced Programming

Assessment WRIT1

Sunrise Dental Clinic

Appointment and Patient Management System

Student Name: U.L. Sithara Ranmal Perera

Cardiff Student ID: st20360408

ICBT Student ID: CL/BSCSD/34/69

Acknowledgement

I would like to express my sincere gratitude to Ms. Priyanga, Lecturer in Advanced Programming, for the invaluable guidance, support and encouragement given throughout this module. The teaching on design patterns, layered architecture and test-driven development shaped almost every decision in this project.

The habit of asking which class should own a rule before writing any code came directly from those sessions, and it is the single thing that kept this system tidy as it grew from a handful of classes into a complete three-tier application.

I am also grateful to ICBT Campus and the School of Technologies for providing an excellent academic environment, and to Cardiff Metropolitan University for the structure of the programme, which encouraged me to think about professional standards rather than only about getting the program to run.

I would further like to thank the open-source community. This project stands on work I did not have to write myself: the Spring Boot, Hibernate and Flyway projects, the JUnit 5, Mockito and AssertJ maintainers, and the H2 and MySQL teams.

Finally, I thank my family and my classmates for their patience and encouragement during the completion of this assignment.

Table of Contents

Table of Contents	3
Task A	6
1. Use Case Diagram.....	6
1.1 Receptionist Use Cases	6
1.2 Dentist Use Cases	7
1.3 Administrator Use Cases.....	7
1.4 System Functionalities and Relationships	8
1.5 Assumptions Made.....	8
2. Class Diagram.....	11
2.1 Access Modifiers	11
2.2 Relationships.....	11
3. Sequence Diagrams.....	13
3.1 Sign In Sequence Diagram.....	13
3.2 Register Appointment Sequence Diagram.....	13
3.3 Calculate Bill Sequence Diagram	13
Task B	17
1. System Architecture.....	17
2. User Authentication (Login)	18
2.1 Overview	18
2.2 Functional Flow	18
2.3 Login Page (login.html).....	18
2.4 Security Configuration (SecurityConfig.java).....	18
2.5 User Details Service (ClinicUserDetailsService.java).....	18
2.6 Audit Listener (AuthenticationAuditListener.java)	19
2.7 Security Features.....	19
3. Register New Appointment.....	20

3.1 Overview	20
3.2 Functional Flow	20
3.3 Booking Form (appointments/form.html).....	20
3.4 Appointment Service (AppointmentService.java)	20
3.5 Validation Chain (BookingValidationChain.java).....	21
3.6 Appointment Repository (AppointmentRepository.java)	21
3.7 Security Features.....	22
4. Display Appointment Details.....	22
4.1 Overview	22
4.2 Functional Flow	22
4.3 Search and Detail Pages.....	22
4.4 REST Endpoint (AppointmentApiController.java)	22
4.5 Recently Viewed Trail (RecentlyViewedTracker.java).....	22
5. Calculate and Print Bill.....	23
5.1 Overview	23
5.2 Functional Flow	23
5.3 Billing Pages (billing/calculate.html, billing/receipt.html).....	23
5.4 Billing Service (BillingService.java)	23
5.5 Pricing Strategy Factory (PricingStrategyFactory.java)	24
5.6 Abstract Pricing Strategy (AbstractPricingStrategy.java)	24
5.7 Security Features.....	24
6. Reports and Help.....	25
6.1 Overview	25
6.2 Functional Flow	25
6.3 Report Generators (service/report)	25
6.4 Reporting DAO (JdbcReportingDao.java).....	25
6.5 Help Service (HelpService.java).....	25

7. Database Design.....	26
7.1 How Double Booking Is Actually Prevented.....	26
7.2 Advanced Database Features	27
8. Validation Mechanisms	27
9. Sessions and Cookies.....	28
Task C	31
1. Test Rationale and Test-Driven Development.....	31
2. Test Data	32
3. Test Plan.....	32
4. Test Cases	33
5. Test Automation and Results	39
6. Defects Found and Fixed	39
7. Traceability	40
Task D.....	41
1. GitHub Repository	41
2. Version Control Techniques	41
3. Workflows Deployed with the Repository	41
4. Evaluation	42
References.....	43

Task A

This section presents the UML design of the Sunrise Dental Clinic appointment and patient management system. It covers the use case diagram, the class diagram and three sequence diagrams, together with the design decisions and the assumptions behind them.

1. Use Case Diagram

Three members of staff use the system. Patients do not sign in, because the assessment brief states that only authorised staff may use it. Two external systems also appear as actors: the e-mail and SMS gateway, and the scheduler that starts the daily reminder run.

1.1 Receptionist Use Cases

The receptionist has the following interactions with the system:

Login

- The receptionist enters a username and password.
- The system validates the credentials against the stored BCrypt hash.
- (Includes) Validate Credentials.
- (Extends) Lock Account After 5 Failed Attempts.

Register New Appointment

- The receptionist selects a patient, a dentist, a treatment, a date and a time.
- The system issues the appointment number automatically, so it can never be mistyped.
- (Includes) Check Slot Availability and Send Confirmation Message.
- (Extends) Register New Patient, when the patient is not already on file.

Display Appointment Details

- The receptionist searches by appointment number, patient name or telephone number.
- The system shows the complete patient and appointment information.
- (Includes) Search By Appointment Number.
- (Extends) Reschedule Appointment.

Calculate and Print Bill

- The system calculates the total from the consultation fee and the treatment cost.
- (Includes) Apply Pricing Rule and Calculate VAT and Discount.
- The printable receipt is produced by the server, so the browser and the desktop client print identically.

Record Payment

- The receptionist records a full or partial payment.
- (Extends) Print Receipt.

Help

- Step-by-step instructions are provided for new staff.

Logout

- Ends the session securely and releases it on the server.

1.2 Dentist Use Cases

The dentist has a deliberately narrower set of interactions:

Login

- The dentist signs in with their own account, which is linked to their dentist code.

Display Appointment Details

- The dentist views the diary and the patient history.

Update Visit Status

- The dentist marks a visit as confirmed, completed, cancelled or a no-show.
- (Includes) Write Audit Log, so every change is recorded.
- Only a completed visit becomes billable.

Logout

- Ends the dentist session.

1.3 Administrator Use Cases

Login

- The administrator signs in with full privileges.

View Reports

- Five management reports are available.
- (Includes) Read Reporting Views, so a report and an ad-hoc SQL query cannot disagree.

Calculate and Print Bill

- The administrator can also issue and reprint bills.

Logout

- Ends the administrator session.

1.4 System Functionalities and Relationships

- All three roles share one sign-in screen; what they may do afterwards is decided by their role.
- <<include>> is used where the behaviour happens every time - a booking always checks slot availability, and a bill always applies exactly one pricing rule.
- <<extend>> is used where the behaviour is conditional - a new patient is registered only when the patient is not already on file, and an account is locked only after five failed attempts.
- Booking and billing are connected: only a completed appointment can be billed, and each appointment can be billed once.
- The system secures access by validating credentials and auditing every attempt with the caller's IP address.

1.5 Assumptions Made

#	Assumption	Reason
A1	Only staff use the system; patients do not log in.	The brief states that only authorised staff may use it.
A2	Three roles: Receptionist, Dentist, Administrator.	A single shared login would make the audit trail meaningless.
A3	One appointment is one visit, one dentist, one treatment.	Keeps the appointment number a genuine unique identifier.
A4	Fixed 30-minute slots between 08:00 and 20:00.	The brief gives no clinic hours; a fixed grid makes "is this slot free?" a question with one correct answer.
A5	Only a completed visit can be billed.	Billing a visit that has not happened is exactly the billing error the clinic wants to escape.
A6	Prices are in Sri Lankan Rupees and VAT is 15%.	The clinic is in Colombo. The rate is one constant.

#	Assumption	Reason
A7	Cancelling a visit releases its slot immediately.	A slot held by a cancelled visit cannot be sold.

The complete use case diagram is shown in Figure 1.

Sunrise Dental Clinic - Use Case Diagram

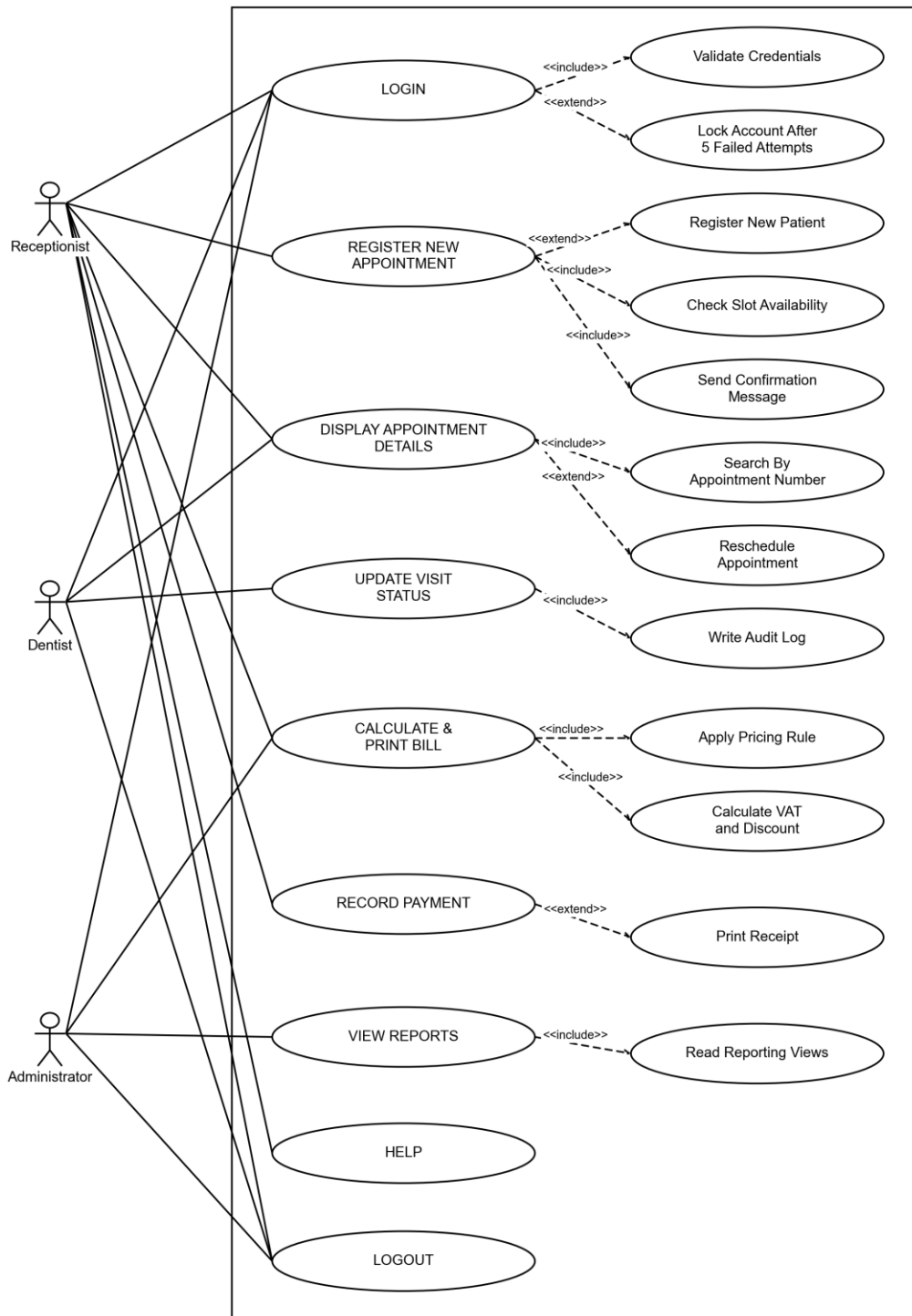


Figure 1 - Use Case Diagram

2. Class Diagram

The class diagram in Figure 2 is drawn from the classes that were actually written; the attribute names, types and method signatures are the ones in the source code. Getters and setters are left out, because listing them would bury the methods that carry the business rules.

2.1 Access Modifiers

- Every attribute is **private** and shown with a minus sign.
- Every business method is **public** and shown with a plus sign.
- The four fields inherited from `BaseEntity` are **protected** and shown with a hash.
- `syncSlotLock()` is deliberately private. It maintains the field that prevents double booking, and a slot lock that disagreed with the appointment date would silently break that safeguard.

2.2 Relationships

Relationship	Type	Meaning
Patient - Appointment	Aggregation (1 to 0..*)	A patient books many appointments. Deleting a patient must not silently delete the clinical record of the visits they attended.
Invoice - InvoiceLine	Composition (1 to 1..*)	A line has no meaning apart from its invoice, so the parts die with the whole. The foreign key carries ON DELETE CASCADE.
Appointment - Dentist	Association (0..* to 1)	Navigable one way only. A two-way link would drag a dentist's whole history into memory on every load.
Appointment - Treatment	Association (0..* to 1)	The treatment carries the pricing strategy key used when billing.
Appointment - Invoice	Association (1 to 0..1)	Zero while the visit is scheduled, one once it is billed. Enforced by a unique key so a visit cannot be billed twice.
PricingStrategy hierarchy	Generalisation	Four concrete rules extend one abstract class whose <code>calculate()</code> method is final.

The contrast between aggregation and composition is the important one. Both read as "has many" in ordinary speech and both would be a List in Java, but they behave differently when something is deleted, and the class diagram is where that difference is decided.

Sunrise Dental Clinic - Class Diagram

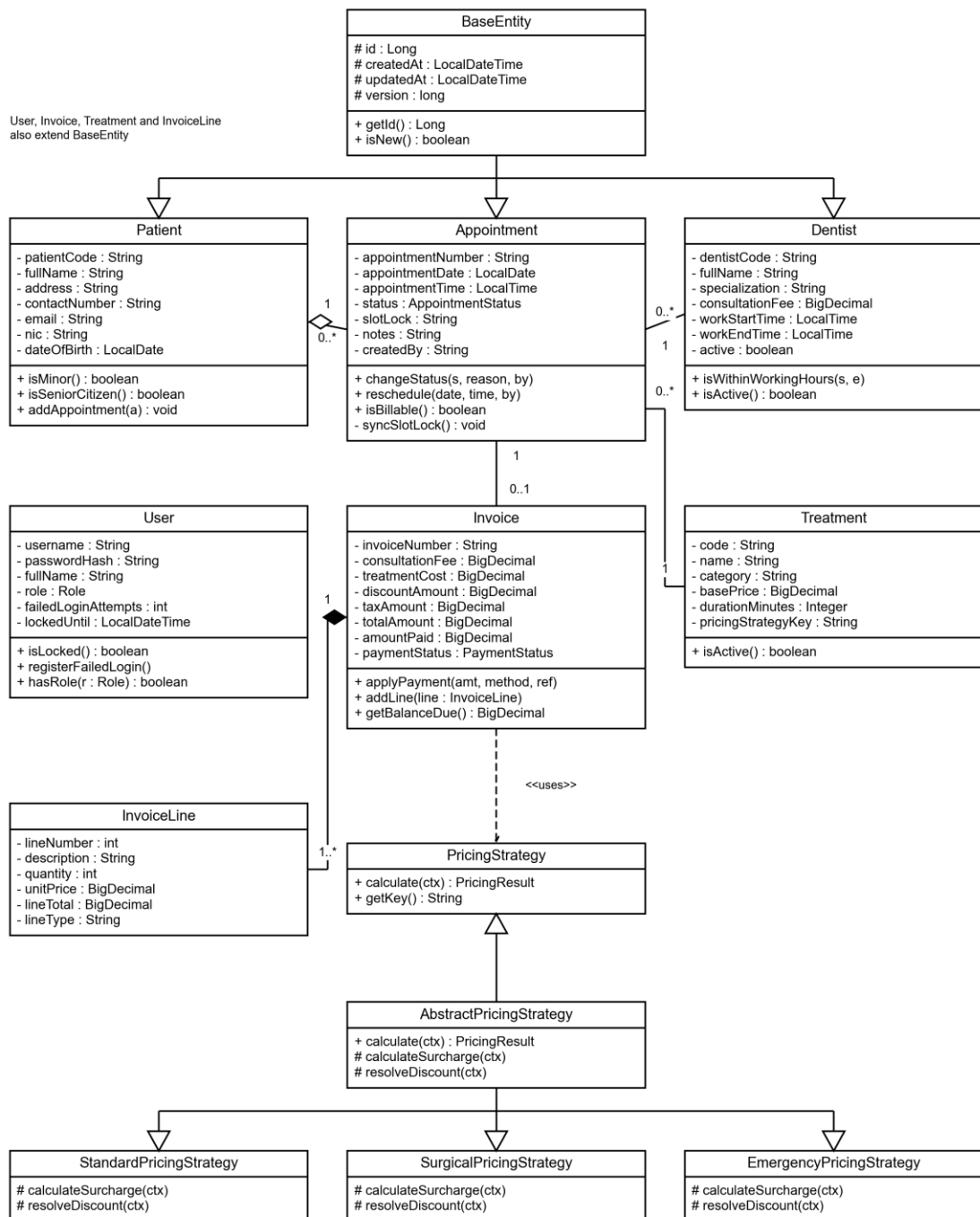


Figure 2 - Class Diagram

3. Sequence Diagrams

Three use cases are shown as sequence diagrams. They were chosen because between them they cover the three hardest parts of the system: security, concurrency and business rules.

3.1 Sign In Sequence Diagram

Figure 3 shows the sign-in flow. An Alternative fragment is used because sign-in has two real outcomes, and both end in the same place: an entry in `audit_log` with the caller's IP address. The same message is shown whether the username exists or not, so the login page cannot be used to discover which accounts are real.

3.2 Register Appointment Sequence Diagram

Figure 4 is the most important diagram in this report. The Loop fragment runs six validation handlers in a fixed order, cheapest first, so a booking for a date in the past is rejected without touching the database at all. The Optional fragment then shows what happens when the check passes but the slot is taken in the few milliseconds before the save: the unique key refuses the insert and the receptionist is shown a sentence rather than a stack trace.

3.3 Calculate Bill Sequence Diagram

Figure 5 shows the Template Method fragment inside `AbstractPricingStrategy`. The six steps always run in that order because the method is final; only two of them are answered differently by each subclass. The diagram also shows the total being checked against the database function `FN_INVOICE_TOTAL` before the invoice is saved.

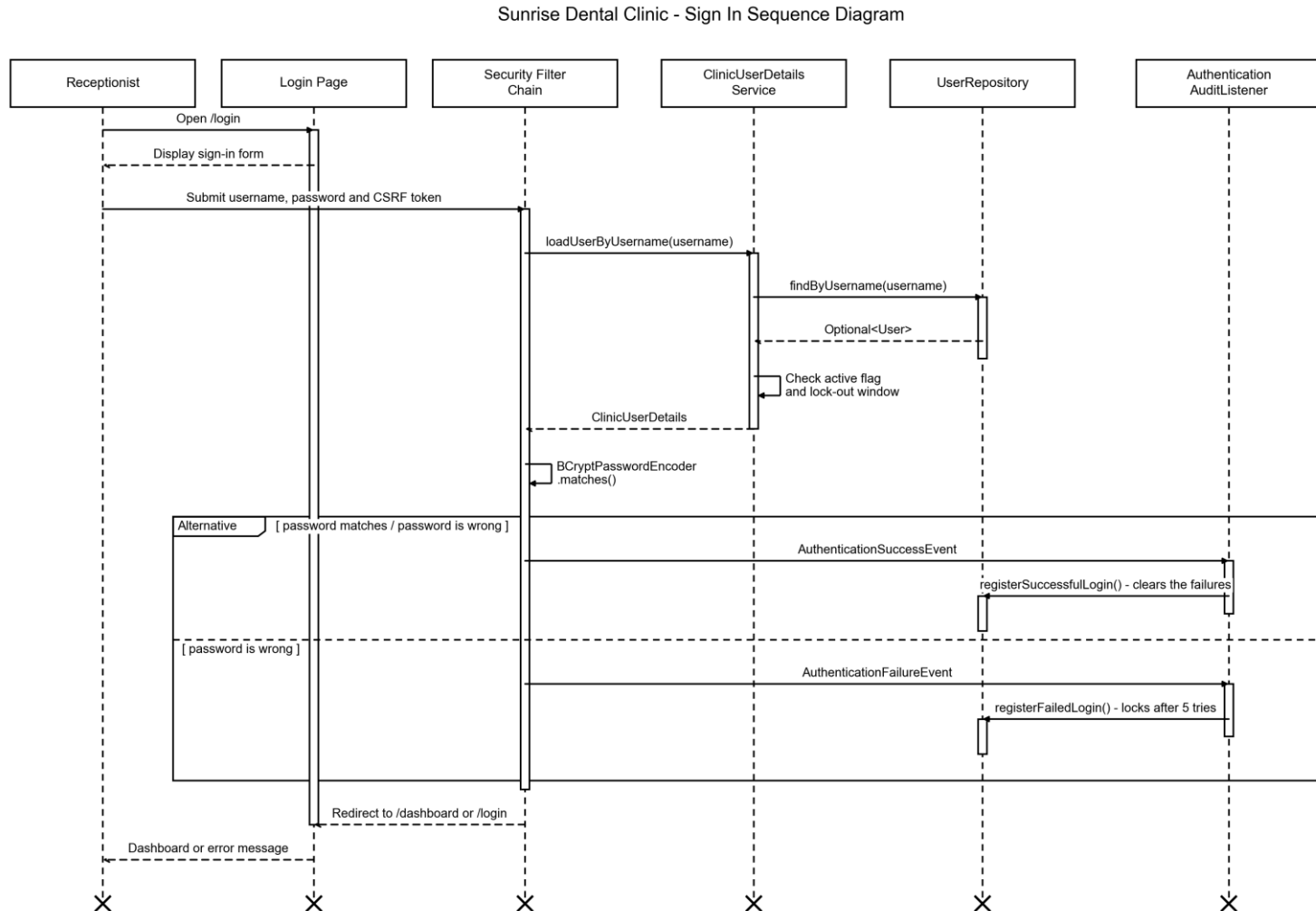


Figure 3 - Sign In Sequence Diagram

Sunrise Dental Clinic - Register Appointment Sequence Diagram

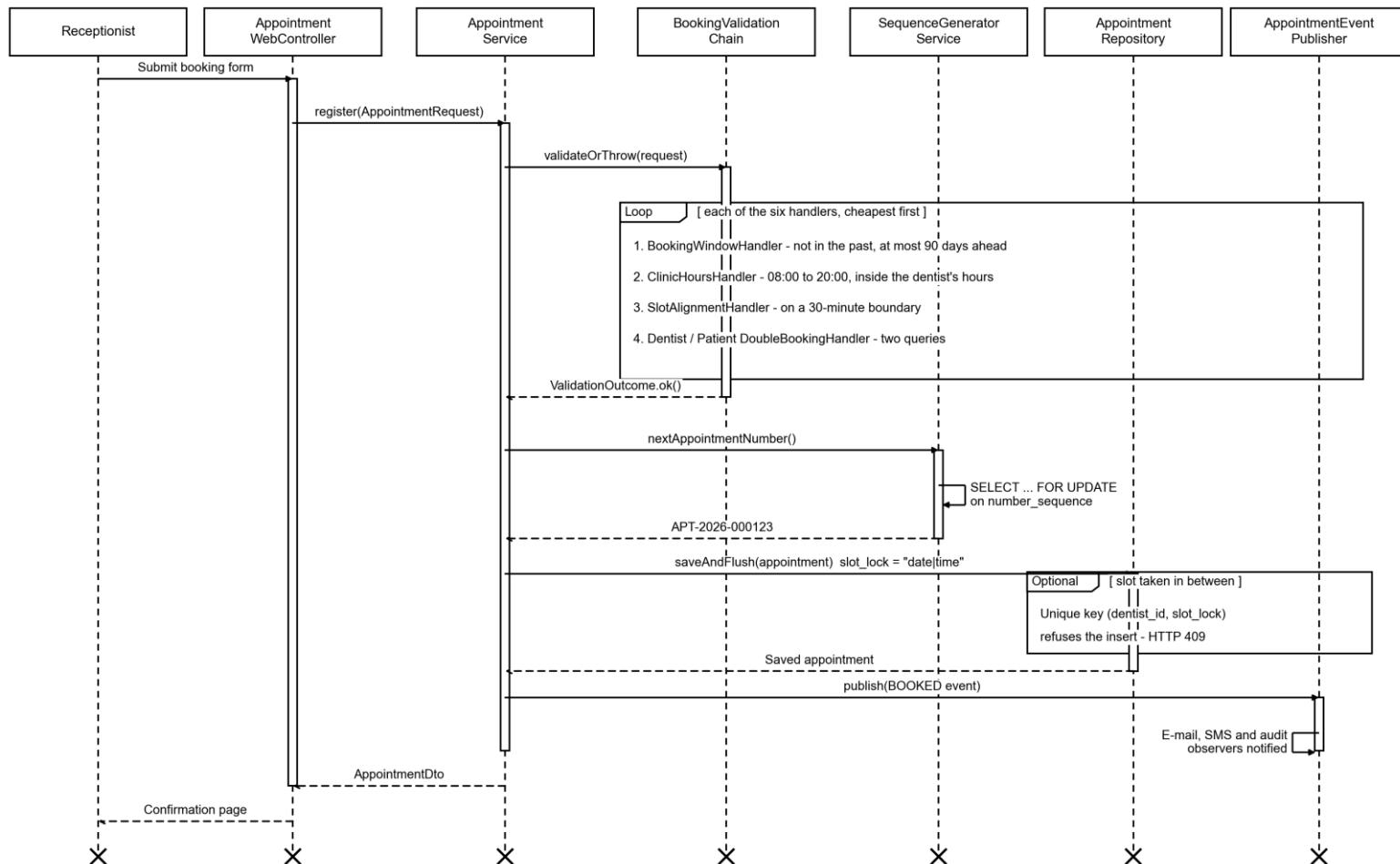


Figure 4 - Register Appointment Sequence Diagram

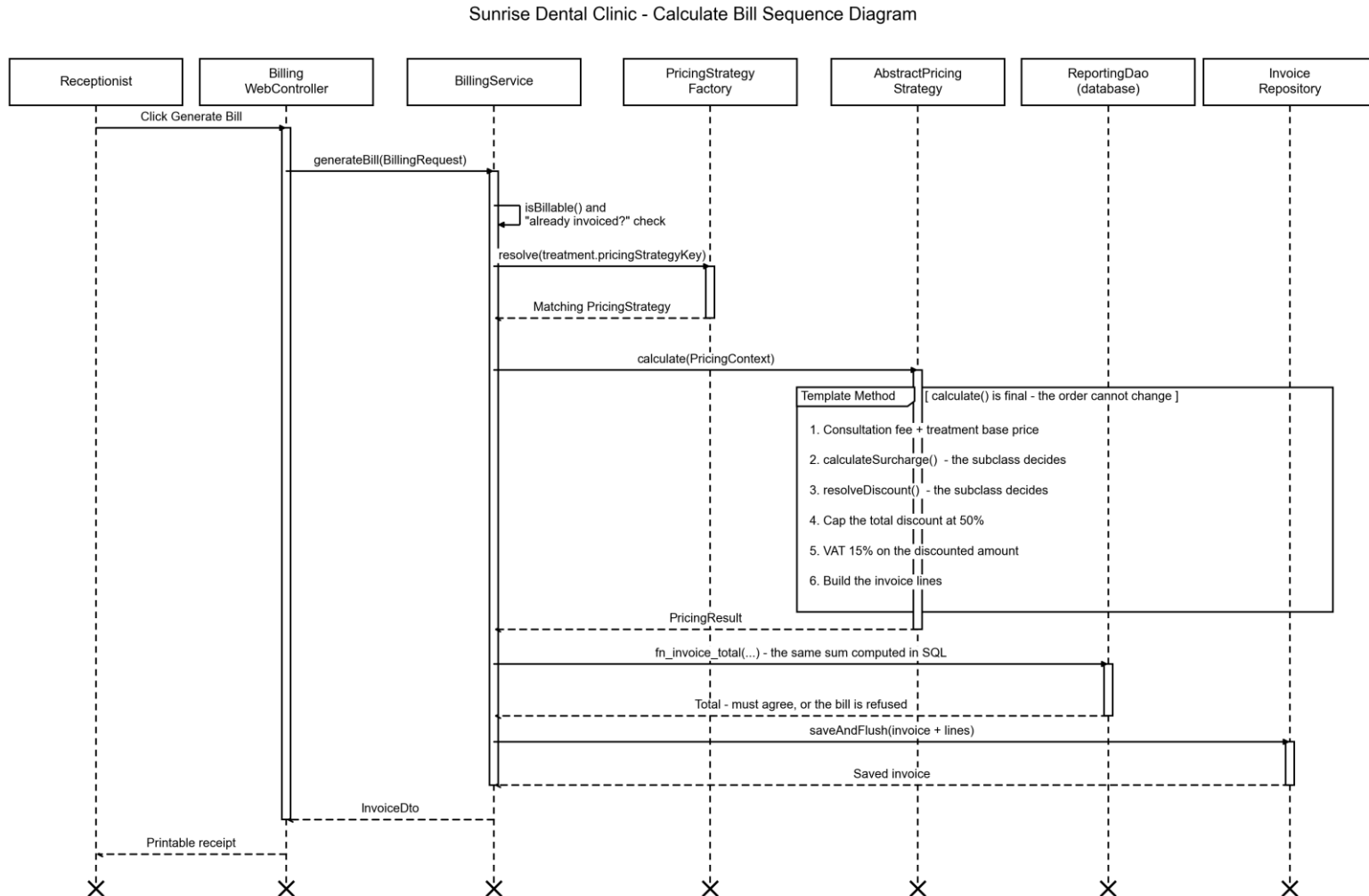


Figure 5 - Calculate Bill Sequence Diagram

Task B

The implementation of the Sunrise Dental Clinic appointment and patient management system follows a structured, modular approach built on a genuine three-tier architecture. This section breaks the system down function by function, explaining the purpose of each, how it is implemented, which design patterns it uses and how it is validated. The system is a distributed application: a Spring Boot server publishes versioned web services, and a separate Java Swing desktop client in its own operating-system process reaches the server over HTTP and JSON only.

1. System Architecture

Figure 6 shows the three tiers. They are separated by a rule that was applied consistently and can be checked: a class may only call downwards. The practical test of whether a layered design is real is whether the top layer can be deleted and the rest still compiles. Here it can, because the desktop client is a different Maven module that does not depend on the server at compile time.

Tier	What lives there	What is not allowed there
Presentation	Thymeleaf controllers, REST controllers, the Swing client, templates and CSS.	Business rules. A controller that decided a discount would put that rule out of reach of the desktop client.
Business	Services, ClinicFacade, pricing strategies, the validation chain, observers and security.	SQL and HTTP. This tier does not know it is reached over the web, which is why the same rules serve both front ends.
Data	Spring Data repositories, ReportingDao, Flyway migrations, views, functions, procedures and triggers.	Decisions. The data tier stores and constrains; it does not choose what the clinic should do.

Three Maven modules make the build mirror the architecture. Because the client depends only on the shared contract module, reaching into a server class from the client is impossible - the compiler stops it. Architecture enforced by the build is worth more than architecture enforced by good intentions.

2. User Authentication (Login)

2.1 Overview

The login function allows staff to authenticate into the system with a username and password. Passwords are never stored or compared as plain text; only a BCrypt hash is held. The process follows a layered architecture, so the controller, the service and the data access are cleanly separated.

2.2 Functional Flow

- The user enters credentials on the Thymeleaf login page.
- Spring Security's filter chain intercepts the POST and reads the credentials together with the CSRF token.
- ClinicUserDetailsService loads the account through UserRepository.
- The account's active flag and lock-out window are checked.
- BCryptPasswordEncoder compares the supplied password with the stored hash.
- On success the session is created and the user is redirected to the dashboard; on failure the same generic error is shown.

2.3 Login Page (login.html)

A Thymeleaf form that posts the username, password and CSRF token. HTML5 required attributes catch empty fields without a round trip, and an opt-in remember-me checkbox is offered.

2.4 Security Configuration (SecurityConfig.java)

Declares two separate filter chains, ordered so that `/api/**` is matched first. An API caller receives a JSON 401 while a browser user is redirected to the login page. A single chain would have sent HTML login pages to the desktop client.

2.5 User Details Service (ClinicUserDetailsService.java)

Loads the account by username and wraps it in a ClinicUserDetails object carrying the role. It reports the same failure for an unknown username as for a wrong password.

2.6 Audit Listener (AuthenticationAuditListener.java)

Listens for Spring Security's success and failure events and writes an audit_log row for each, including the caller's IP address. The bookkeeping update runs in its own transaction so that two simultaneous sign-ins cannot corrupt each other.

Design Patterns Used

- **MVC Pattern:** the web controller handles the request and delegates to the service layer.
- **Singleton:** Spring manages one instance of each security bean.
- **Observer:** the audit listener reacts to authentication events rather than being called directly.

OOP Concepts Used

- **Encapsulation:** the password hash is private and there is no getter that returns it in a usable form.
- **Abstraction:** hashing is hidden behind PasswordEncoder, so no caller knows which algorithm is used.
- **Polymorphism:** ClinicUserDetails implements Spring Security's UserDetails interface.

2.7 Security Features

- **Password Hashing:** BCrypt with a per-password salt. Plain text is never stored or logged.
- **Account Lock-out:** five consecutive failures lock the account for fifteen minutes.
- **Session Management:** an HTTP session holds the signed-in user; sign-out invalidates it on the server.
- **CSRF Protection:** a token is required on every form, so a stolen session cookie alone cannot submit one.
- **Uniform Error Message:** the same message is shown whether the username exists or not, so the page cannot be used to discover valid accounts.

3. Register New Appointment

3.1 Overview

This is the core function of the system and the one that answers the clinic's worst problem. It collects the patient, dentist, treatment, date and time, validates the booking against six independent rules, issues a unique appointment number and saves the record. The appointment number is generated by the server, so there is deliberately no field to type it into.

3.2 Functional Flow

- The receptionist submits the booking form.
- AppointmentWebController passes an AppointmentRequest to AppointmentService.
- The patient is matched by code, NIC or name and telephone, or registered once if new.
- BookingValidationChain applies six rules in order, cheapest first.
- SequenceGeneratorService issues the next appointment number under a pessimistic row lock.
- The appointment is saved with slot_lock set, and a unique database constraint has the final word on double booking.
- An event is published, and the e-mail, SMS and audit observers react independently.

3.3 Booking Form (appointments/form.html)

The dentist, treatment and slot are chosen from reference data rather than typed, so an invalid combination cannot be entered. Free slots are offered up front, so a taken slot is never even requested.

3.4 Appointment Service (AppointmentService.java)

Coordinates the whole operation inside one transaction: resolve the reference data, validate, allocate a number, save, publish. It catches the database's unique-constraint violation and translates it into a sentence a receptionist can act on.

3.5 Validation Chain (BookingValidationChain.java)

Six handlers assembled in a fixed order, logged at start-up: booking window, clinic hours, slot alignment, dentist availability, dentist double booking and patient double booking. The first three need no database access, so a booking for last Tuesday is rejected without a single query, and the chain short-circuits on the first failure.

3.6 Appointment Repository (AppointmentRepository.java)

A Spring Data interface. The save is guarded by `uk_appointment_slot` on (`dentist_id`, `slot_lock`), which is what actually prevents two bookings for one chair.

Design Patterns Used

- **Chain of Responsibility:** each booking rule is an independent handler that can be tested on its own and reordered without touching the others.
- **Builder:** Appointment.Builder makes it impossible to create a half-built appointment with an inconsistent slot lock.
- **Observer:** AppointmentEventPublisher notifies the e-mail, SMS and audit observers and isolates their failures.
- **Facade:** ClinicFacade collapses multi-call sequences into one round trip for the remote desktop client.

OOP Concepts Used

- **Encapsulation:** slotLock is private and maintained only by the class that owns it.
- **Inheritance:** every entity extends BaseEntity, which supplies the identifier, timestamps and version column.
- **Polymorphism:** the chain calls handle() on each handler without knowing which rule it is.

Data Structures and Algorithms Used

- **Short-circuit evaluation:** the chain stops at the first failure, so the user is told the most fundamental problem.
- **Ordered cost-based execution:** the two handlers that cost a database query run last.
- **Pessimistic locking:** SELECT ... FOR UPDATE on number_sequence, so two receptionists cannot be issued the same appointment number.

3.7 Security Features

- **Server-issued identifiers:** the client can never choose an appointment number.
- **Bean Validation:** constraints travel with the DTOs, so the desktop client enforces identical rules with no duplicated code.
- **Database constraints:** CHECK and UNIQUE constraints hold even against a direct SQL statement.

4. Display Appointment Details

4.1 Overview

The receptionist or dentist searches for an appointment and sees the complete patient and appointment information. The brief asks for search by appointment number; search by patient name and telephone number was added because a patient at the counter rarely remembers their reference.

4.2 Functional Flow

- The user enters an appointment number, a name or a telephone number.
- AppointmentService looks the record up and returns a DTO.
- The page shows the patient, dentist, treatment, date, time, status and any invoice.
- A session-scoped trail remembers recently viewed appointments.
- If nothing matches, a clear message is shown rather than an empty table.

4.3 Search and Detail Pages

appointments/list.html and appointments/view.html. The detail page shows the appointment's full history and offers the actions the signed-in role is allowed to take.

4.4 REST Endpoint (AppointmentApiController.java)

GET /api/v1/appointments/{number} returns the same data as the web page inside the standard response envelope, so the desktop client and the browser can never disagree.

4.5 Recently Viewed Trail (RecentlyViewedTracker.java)

Stored in the HTTP session only. It exists so a receptionist interrupted mid-task can get back to where they were, and it is deliberately not persisted to the database.

Design Patterns Used

- **DTO:** no JPA entity ever crosses the network.
- **Repository:** query methods are declared on an interface; no SQL appears in the service.

OOP Concepts Used

- **Encapsulation:** the session trail is only reachable through its own component.

5. Calculate and Print Bill

5.1 Overview

Billing calculates the total from the consultation fee and the treatment cost, applies the correct pricing rule, caps any discount, adds VAT and produces a printable receipt. The client can never send a price: every figure comes from the dentist and treatment records held on the server.

5.2 Functional Flow

- The receptionist opens a completed appointment and clicks Generate Bill.
- BillingService checks that the visit is billable and has not already been invoiced.
- PricingStrategyFactory resolves the rule named by the treatment's pricing strategy key.
- AbstractPricingStrategy.calculate() runs the six fixed steps.
- The Java total is cross-checked against the database function FN_INVOICE_TOTAL.
- The invoice and its lines are saved, and the receipt is rendered by the server.

5.3 Billing Pages (billing/calculate.html, billing/receipt.html)

Calculate and issue are deliberately two steps, so the receptionist can see the breakdown and apply a discount before anything is committed.

5.4 Billing Service (BillingService.java)

Refuses to bill a visit that has not happened, and refuses to bill the same visit twice. Both refusals are also enforced by database constraints; the checks exist so the user gets a sentence rather than a constraint violation.

5.5 Pricing Strategy Factory (PricingStrategyFactory.java)

Registers every strategy automatically from the Spring context, so adding a rule cannot mean forgetting to register it. If reference data names a rule that does not exist, it falls back to the standard rule and logs a warning.

5.6 Abstract Pricing Strategy (AbstractPricingStrategy.java)

calculate() is declared final and performs the invariant sequence: fees, surcharge, discount, a 50% cap, VAT, then the invoice lines. Only calculateSurcharge() and resolveDiscount() are abstract, so a subclass decides how much but never in what order.

Design Patterns Used

- **Strategy:** four genuinely different billing policies - standard, surgical, cosmetic and emergency. A switch statement would put every future rule inside the one method that prints every bill in the clinic.
- **Template Method:** the order of the billing steps is fixed by tax law and is final; only the two variable steps are overridable.
- **Factory:** turns a database value into an object without any caller naming a concrete class.

OOP Concepts Used

- **Abstraction:** BillingService never learns which pricing rule ran.
- **Inheritance:** the four rules share the invariant sequence from one abstract parent.
- **Polymorphism:** one call site invokes whichever rule the treatment names.

Data Structures and Algorithms Used

- **Ordered arithmetic:** VAT is applied after the discount, because VAT on a discounted amount differs from VAT on an undiscounted one.
- **Bounded value:** the total discount is capped at 50% before tax is computed.
- **Cross-verification:** the same total is computed independently in Java and in SQL and the two are compared.

5.7 Security Features

- **No client-supplied prices:** fees come from the dentist and treatment records only.

- **Immutable snapshot:** every figure is stored on the invoice, so a historic receipt does not change when the price list does.
- **Payment ceiling:** a CHECK constraint prevents a payment greater than the total.

6. Reports and Help

6.1 Overview

Five management reports support decisions the clinic actually has to make, and a help section gives new staff step-by-step instructions as the brief requires.

6.2 Functional Flow

- The administrator opens the reports page and chooses a report and a date range.
- ReportGeneratorFactory resolves the requested report.
- The generator reads a database view rather than assembling joins in Java.
- The result is returned in one shared envelope, so a single page displays any report.
- The help topics are served from the same API the desktop client uses, so both front ends always agree.

6.3 Report Generators (service/report)

Each generator extends AbstractReportGenerator, which fixes the shape of every report. Adding a sixth report requires no user interface work at all.

6.4 Reporting DAO (JdbcReportingDao.java)

A hand-written JDBC DAO for reporting reads, which aggregate thousands of rows and belong in SQL rather than in Java loops.

6.5 Help Service (HelpService.java)

Eight topics and forty-two numbered steps, served from one place so the browser and the desktop client cannot drift apart.

Design Patterns Used

- **Template Method:** the report skeleton is fixed; only the query and the row mapping vary.
- **Factory:** report generators register themselves.

- **DAO:** reporting SQL is isolated from the business tier.

Report	Question it answers	Decision it supports
Daily Schedule	Who is coming in today, and to which dentist?	Staffing the front desk and preparing the surgeries.
Revenue	What did the clinic take, by day and by treatment?	Whether a treatment is worth continuing to offer.
Outstanding Invoices	Who owes money, and for how long?	Which debts to chase first; the ageing bands come from FN_AGEING_BUCKET.
Dentist Workload	How busy is each dentist?	Rebalancing the rota and deciding whether to recruit.
Treatment Popularity	Which treatments are most in demand?	Stock ordering and continuing-education choices.

7. Database Design

Figure 7 shows the physical schema: nine tables, five views, two stored functions, two stored procedures and four triggers. The schema is owned by versioned Flyway migration scripts and Hibernate runs with `ddl-auto=validate`, so the application refuses to start if the Java entities and the real schema disagree.

7.1 How Double Booking Is Actually Prevented

The validation chain checks whether a slot is free, but between that check and the insert there is a window of a few milliseconds in which another transaction can commit the same slot. Any solution that lives only in Java has this window.

The solution is a column called `slot_lock` and a unique constraint on (`dentist_id`, `slot_lock`). While an appointment occupies a chair, `slot_lock` holds the string "date|time"; when the visit is cancelled or missed it is set back to NULL. Because SQL never treats two NULLs as equal, any number of cancelled appointments can share a dentist and slot, while two live ones cannot. A database constraint has no race window.

This was verified rather than assumed. Twelve genuinely simultaneous booking requests for the same dentist, date and time produced exactly one success and eleven clean HTTP 409 responses, repeated against three fresh databases.

7.2 Advanced Database Features

Feature	Name	Business rule it implements
Function	FN_INVOICE_TOTAL	Recomputes an invoice total in SQL. Every bill is checked against it, so a rounding mistake in Java is caught first.
Function	FN_AGEING_BUCKET	Puts a debt in a 0-30 / 31-60 / 61-90 / 90+ day band. The report calls it, so it cannot disagree with an ad-hoc query.
Procedure	SP_SETTLE_INVOICE	Applies a payment and updates the status in one atomic operation.
Procedure	SP_DAILY_CLOSING_SUMMARY	Produces the end-of-day figures needed to cash up.
Trigger	trg_appointment_before_insert	Refuses a booking outside the dentist's working hours, even from a direct SQL insert (MySQL).
Triggers	trg_appointment_audit, trg_invoice_payment_audit	Write the audit trail at database level, so a change made outside the application is still recorded.
Views	5 reporting views	Reports read views, so the definition of revenue lives in one place.
Constraints	FK, UNIQUE and CHECK on every table	Discounts cannot exceed 50%, payments cannot exceed the total, and a status must be one of the permitted values.

Two database engines are supported from one codebase: H2 in file mode by default, so the application runs with nothing to install, and MySQL 8 through a Spring profile. The MySQL path was tested end to end against a real server; all four migrations, both functions, both procedures and every trigger were exercised with real data.

8. Validation Mechanisms

Validation is applied in four places, on the principle that each layer should refuse whatever it can prove is wrong.

Layer	Mechanism	Example
Browser	HTML5 required, type and pattern attributes	An empty patient name is caught without a round trip.
Shared DTO	Bean Validation annotations in dental-common	A telephone number must match the Sri Lankan format. The desktop client enforces the identical rule with no duplicated code.
Business tier	The six-handler booking validation chain	A booking outside the dentist's own working hours is refused.
Database	CHECK, UNIQUE and FOREIGN KEY constraints, triggers	A discount above 50% is refused even from a SQL console.

Messages are written for a receptionist rather than a developer. Instead of a constraint violation, the user is told which slot is taken and what to do about it, while the response envelope carries a stable error code so the desktop client can react programmatically to the same failure.

9. Sessions and Cookies

- **HTTP session:** carries the signed-in user and the recently viewed trail, so a receptionist interrupted mid-task can return to where they were.
- **Remember-me cookie:** opt-in, saving the front-desk machine from signing in again every morning.
- **Preference cookie:** stores the chosen table density for that workstation.
- **CSRF token:** required on every form, so a stolen session cookie alone cannot submit one.

Sunrise Dental Clinic - Three-Tier Architecture

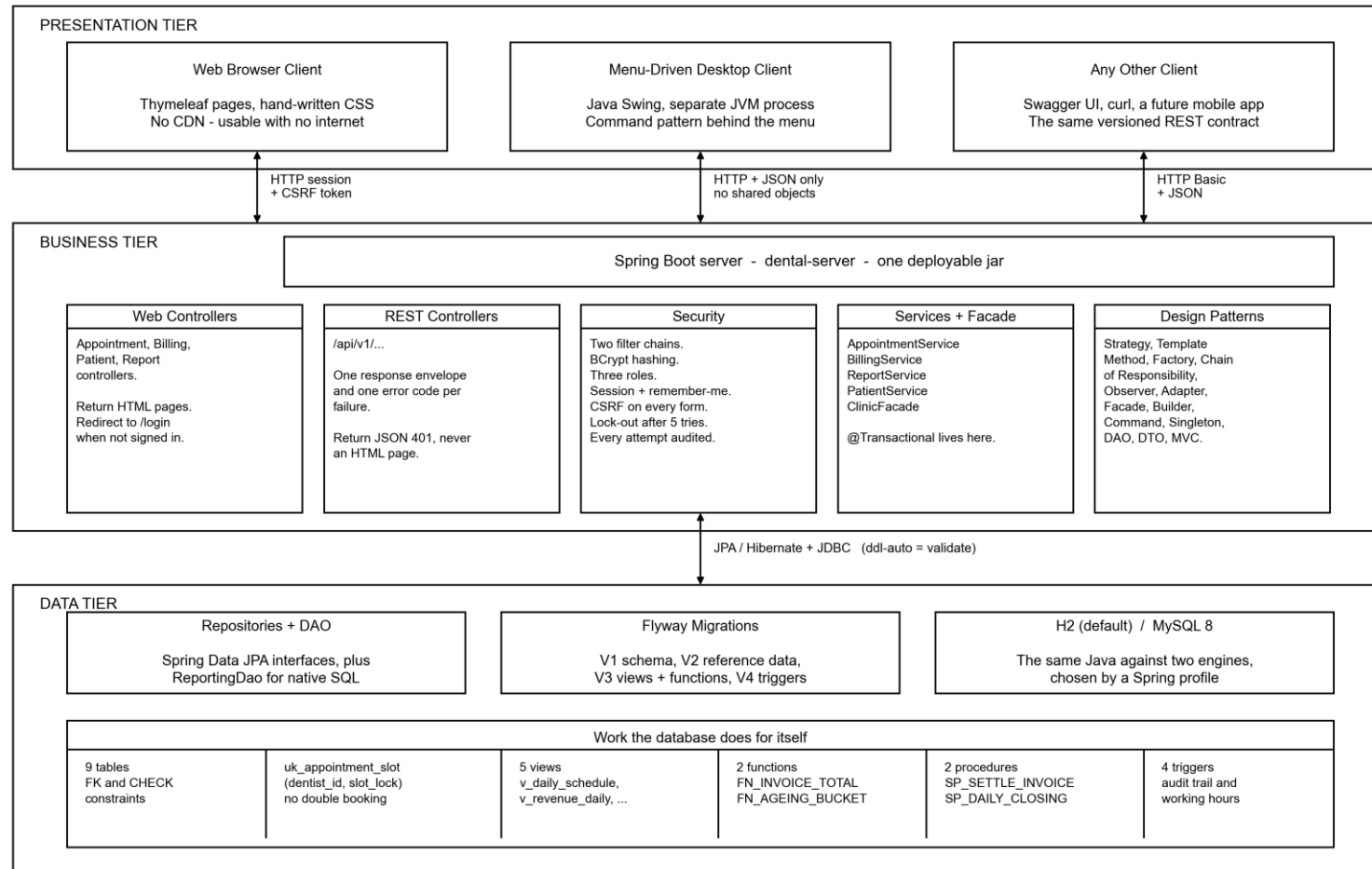
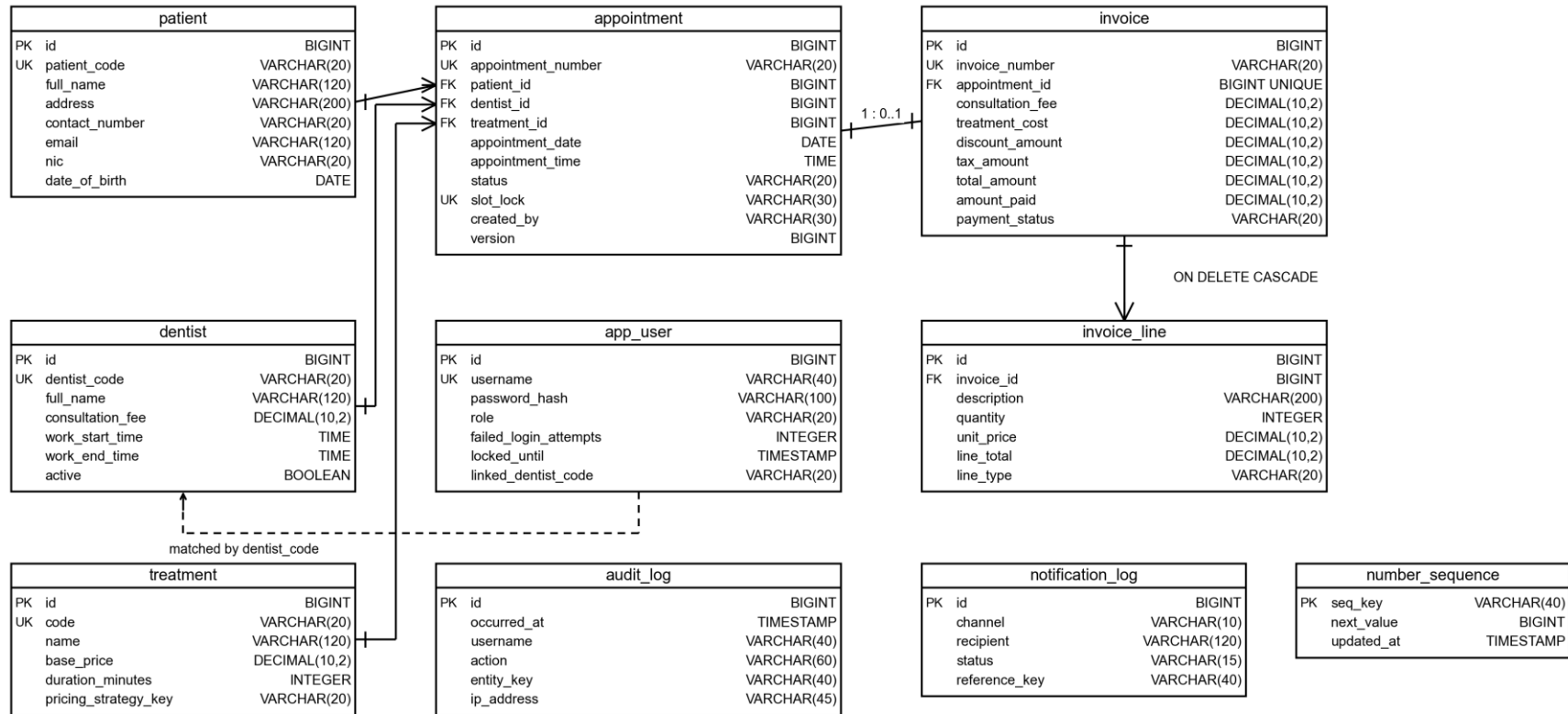


Figure 6 - Three-Tier Architecture

Sunrise Dental Clinic - Entity Relationship Diagram



Also in the schema, not shown as tables: 5 views (v_daily_schedule, v_revenue_daily, v_dentist_workload, v_treatment_popularity, v_outstanding_invoice) | 2 functions (FN_INVOICE_TOTAL, FN_AGEING_BUCKET) | 2 procedures | 4 triggers

Figure 7 - Entity Relationship Diagram

Task C

This section documents the testing approach, the test plan, the test data and the results. The clinic's existing problems are all failures of correctness under pressure, so the suite was designed to prove that the rules hold when they are attacked, not merely that the happy path works.

1. Test Rationale and Test-Driven Development

Testing was carried out at three levels, so that each failure is diagnosed as cheaply as possible.

Level	Count	What it proves
Unit tests	226	One class behaves correctly in isolation. JUnit 5 with Mockito, no Spring context and no database, so a failure points at one class.
Integration tests	97	The parts work together against the real Flyway schema, with real constraints, views, functions and triggers.
Concurrency tests	-	The rules hold under a genuine race, using a purpose-written multi-threaded client.

Test-driven development was applied where it genuinely helps. The billing rules, the appointment lifecycle and the validation chain were written test-first in the red-green-refactor cycle, because they have clear inputs, clear outputs and rules that can be stated in a sentence. Before any code existed, a test asserted that a cancelled appointment cannot become completed; writing that test forced a decision that had been vague until then, and the resulting enum owns the rule so no service can bypass it.

Test-driven development was deliberately not used for the Thymeleaf templates or the Swing windows. Writing a failing test for the position of a form field would lock in a layout before anyone had looked at it. Those parts were built, reviewed by eye, and then covered by integration tests that assert on behaviour rather than on markup.

One case shows the cycle earning its keep. A test asserted that rescheduling an appointment must not report a clash with itself. It failed, as expected; the interesting part was why. The double-booking query excluded nothing, so an appointment was its own conflict. That is a bug a manual tester would have found only after a receptionist reported it.

2. Test Data

Test data was devised to cover boundaries rather than typical values, because a bug almost never lives in the middle of a range.

Category	Values used
Valid, ordinary	A 34-year-old patient, a routine filling at 10:00 next Tuesday, paid in full by card.
Boundary - age	Exactly 18 (not a minor by one day), exactly 60 (senior concession), and 17 and 61 either side.
Boundary - time	08:00 (first legal slot), 20:00 (first illegal), 09:29 and 09:30 either side of the 30-minute grid.
Boundary - money	A discount of exactly 50% (allowed) and 50.01% (refused); a payment equal to the total, and one cent more.
Boundary - dates	Today, yesterday (refused), 90 days ahead (allowed) and 91 (refused).
Invalid	Blank name, nine-digit telephone number, negative price, unknown treatment code, forbidden status transition.
Malicious	A form posted with no CSRF token, a receptionist requesting an administration page, an API call with no credentials.
Concurrent	Twelve threads released simultaneously against one dentist, one date and one time.

3. Test Plan

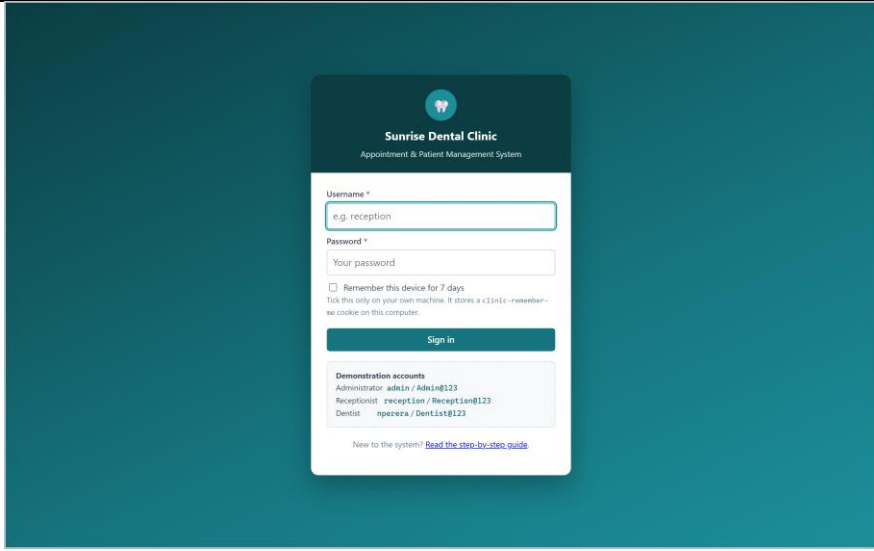
Test ID	Function	Input Data	Expected Result
T-001	User Login (valid)	Username: admin Password: Admin@123	System authenticates the user Displays the dashboard
T-002	User Login (invalid password)	Username: admin Password: wrongPass	System denies access Displays "Invalid username or password"
T-003	Account lock-out	Five consecutive failed sign-in attempts	Account is locked for 15 minutes Further attempts are refused
T-004	Register New Appointment	Patient: Kamala Perera Address: No. 45, Galle Road, Colombo 03 Contact: 0771234567 Dentist: Dr Nimal Perera Treatment: Composite Filling Date: next Tuesday, Time: 10:00	System issues APT-2026-000123 Displays "Appointment registered successfully"
T-005	Register Appointment (missing fields)	Patient name: (empty) Contact: 0771234567	System prevents saving Prompts "Patient name is required"
T-006	Register Appointment (duplicate slot)	Same dentist, same date, same time as an existing booking	System refuses the booking Displays "That slot is no longer available"

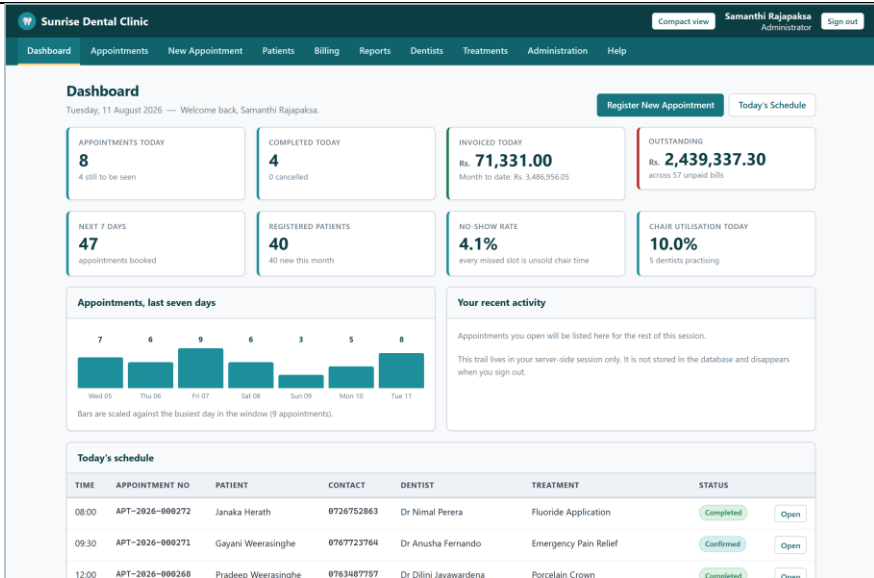
Test ID	Function	Input Data	Expected Result
T-007	Display Appointment Details	Appointment number: APT-2026-000123	System fetches and displays the patient, dentist, treatment, date, time and status
T-008	Display Appointment (unknown number)	Appointment number: APT-9999-999999	System displays "No appointment found" rather than an empty table
T-009	Calculate and Print Bill	Appointment: APT-2026-000123 (completed) Consultation fee: Rs. 2,000.00 Treatment: Rs. 6,200.00 Discount: 10%	Sub-total Rs. 8,200.00, discount Rs. 820.00, VAT 15% Rs. 1,107.00, total Rs. 8,487.00 Printable receipt is produced
T-010	Bill an appointment that is not completed	Appointment: APT-2026-000124 (status SCHEDULED)	System refuses Displays "Only a completed appointment can be billed"
T-011	Bill the same appointment twice	Appointment: APT-2026-000123 (already invoiced)	System refuses Displays the existing invoice number
T-012	Discount above the permitted cap	Discount: 60%	System refuses Discount is capped at 50% by a CHECK constraint
T-013	Record Payment	Invoice: INV-2026-000045 Amount: Rs. 8,487.00, Method: Card	Payment status becomes PAID Balance due becomes Rs. 0.00
T-014	Over-payment	Amount: Rs. 10,000.00 against a Rs. 8,487.00 invoice	System refuses A payment cannot exceed the total
T-015	View Management Reports	Report: Revenue, range: last 30 days	System displays the revenue report read from v_revenue_daily
T-016	Unauthorised access restriction	Signed in as RECEPTIONIST Request /admin/users	System blocks the request Displays "Access Denied"
T-017	Web service endpoint	GET /api/v1/appointments with HTTP Basic credentials	Returns HTTP 200 and the standard JSON envelope
T-018	Web service without credentials	GET /api/v1/appointments with no credentials	Returns HTTP 401 with a JSON body, not an HTML login page
T-019	Concurrent booking of one slot	Twelve simultaneous requests for the same dentist, date and time	Exactly one succeeds Eleven receive HTTP 409 SLOT_UNAVAILABLE
T-020	User Logout	User selects Sign out	Session is invalidated on the server Returns to the login screen

4. Test Cases

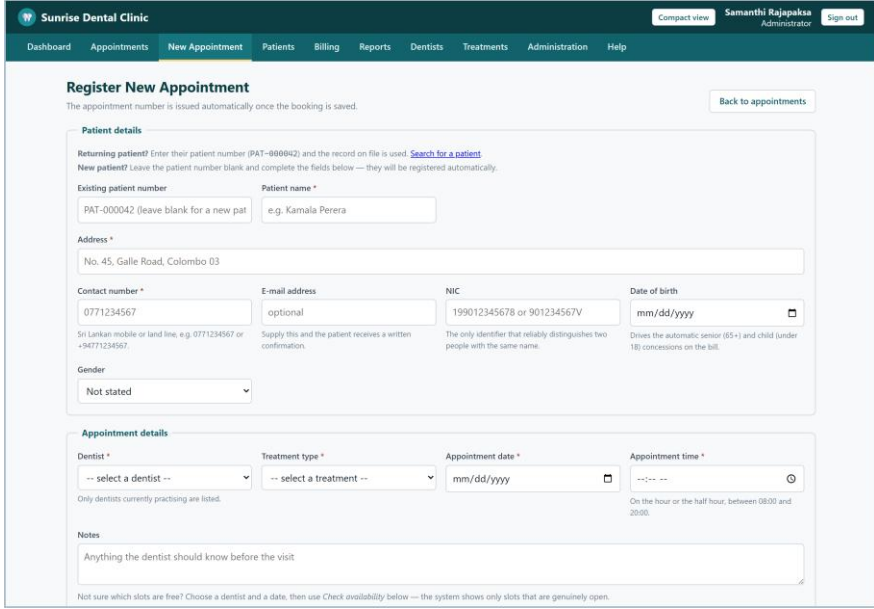
Each test case below was executed against the running system. The screenshot in each table is evidence of the actual result.

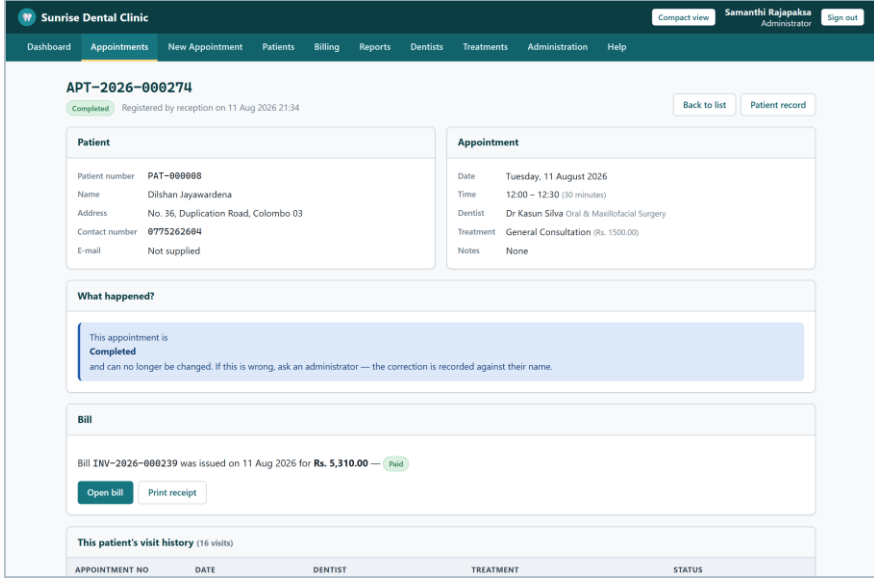
Test Case ID	T-001
Function	User Login (valid)

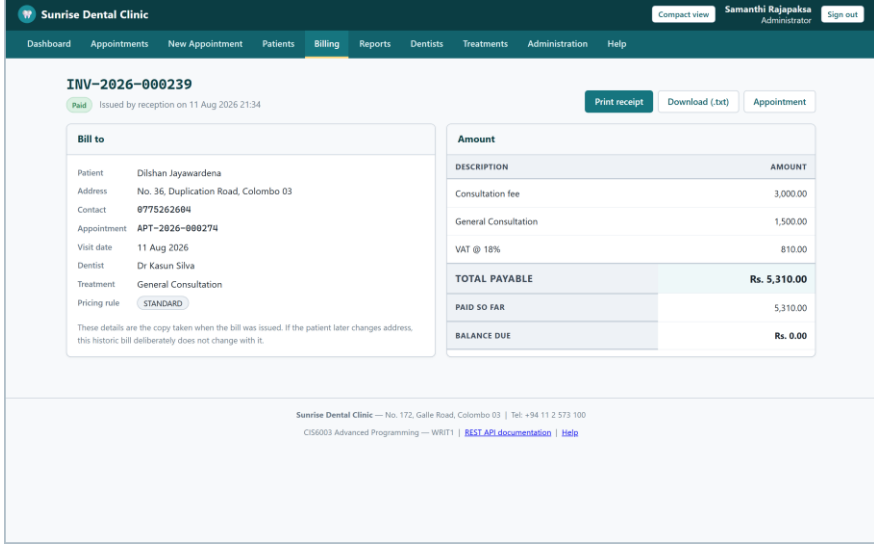
Input Data	Username: admin Password: Admin@123
Expected Result	System authenticates the user and displays the dashboard
Actual Result	Signed in successfully and redirected to the dashboard
Screenshot	

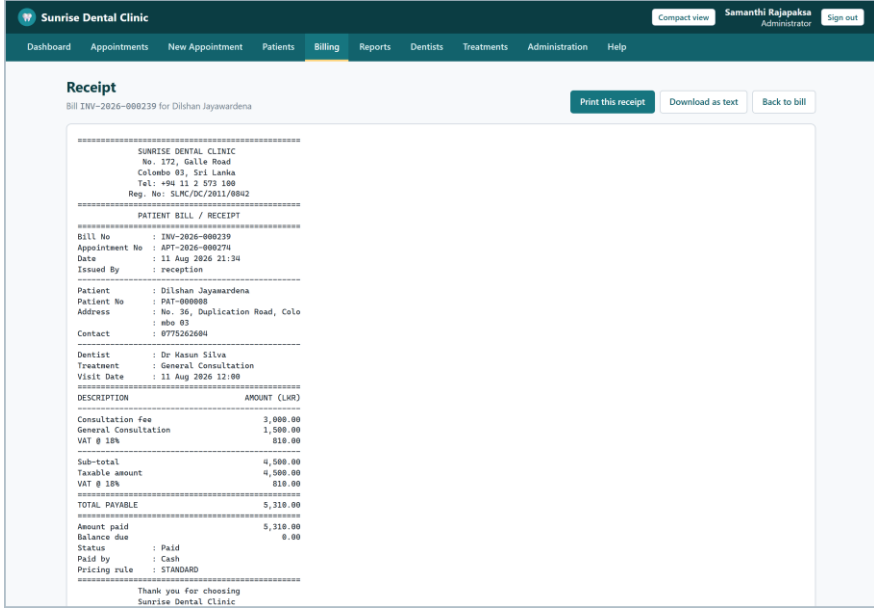
Test Case ID	T-002
Function	Dashboard after sign-in
Input Data	Signed in as admin
Expected Result	Dashboard shows today's appointments, outstanding invoices and revenue
Actual Result	Dashboard displayed with live figures from the seeded data
Screenshot	

Test Case ID	T-004
Function	Register New Appointment

Input Data	Patient: Kamala Perera Dentist: Dr Nimal Perera Treatment: Composite Filling Date: next Tuesday, Time: 10:00
Expected Result	System issues a unique appointment number and confirms the booking
Actual Result	Appointment registered and the number issued automatically
Screenshot	

Test Case ID	T-007
Function	Display Appointment Details
Input Data	Appointment number: APT-2026-000123
Expected Result	System displays the full patient and appointment information
Actual Result	All details displayed correctly, including the linked invoice
Screenshot	

Test Case ID	T-009
Function	Calculate and Print Bill
Input Data	Completed appointment with a 10% discount
Expected Result	System shows consultation fee, treatment cost, discount, VAT and total
Actual Result	Bill calculated correctly and the breakdown displayed
Screenshot	

Test Case ID	T-009b
Function	Print Receipt
Input Data	Invoice: INV-2026-000045
Expected Result	A printable receipt is produced by the server
Actual Result	Receipt rendered; the browser and the desktop client print identically
Screenshot	

Test Case ID	T-015
---------------------	-------

Function	View Management Reports																								
Input Data	Report: Revenue, range: last 30 days																								
Expected Result	System displays the revenue report read from the database view																								
Actual Result	Report displayed with revenue grouped by day and treatment																								
Screenshot	Sunrise Dental Clinic Compact view Samanthi RajapaksaAdministrator Sign out Dashboard Appointments New Appointment Patients Billing Reports Dentists Treatments Administration Help Revenue Analysis Invoiced versus collected income per day, with discounts and VAT separated, so that pricing decisions and collection performance can be judged apart. Print All reports Report period From To mm/dd/yyyy mm/dd/yyyy Generate Dates default to the current month. The maximum range is one year. The debtor ageing report ignores the dates on purpose — a four-month-old unpaid bill is exactly what it exists to surface. Summary Period 01 Aug 2026 to 11 Aug 2026 generated 11 Aug 2026 21:35 by admin 11 rows TRADING DAYS WITH INCOME 11 INVOICES ISSUED 54 GROSS CHARGES Rs. 2,995,950.00 DISCOUNTS GIVEN Rs. 40,902.50 VAT COLLECTED Rs. 531,908.55 TOTAL INVOICED Rs. 3,486,956.05 TOTAL COLLECTED Rs. 3,042,069.50 STILL OUTSTANDING Rs. 444,886.55 COLLECTION RATE 87.2% AVERAGE BILL Rs. 64,573.26 AVERAGE PER DAY Rs. 316,996.00 Revenue Analysis <table><tr><th>DATE</th><th>INVOICES</th><th>GROSS</th><th>DISCOUNT</th><th>VAT</th><th>NET INVOICED</th><th>COLLECTED</th><th>OUTSTANDING</th></tr><tr><td>01 Aug 2026</td><td>6</td><td>123,550.00</td><td>1,052.50</td><td>22,049.55</td><td>144,547.05</td><td>134,402.00</td><td>10,145.05</td></tr><tr><td>02 Aug 2026</td><td>2</td><td>324,200.00</td><td>0.00</td><td>58,356.00</td><td>382,556.00</td><td>382,556.00</td><td>0.00</td></tr></table>	DATE	INVOICES	GROSS	DISCOUNT	VAT	NET INVOICED	COLLECTED	OUTSTANDING	01 Aug 2026	6	123,550.00	1,052.50	22,049.55	144,547.05	134,402.00	10,145.05	02 Aug 2026	2	324,200.00	0.00	58,356.00	382,556.00	382,556.00	0.00
DATE	INVOICES	GROSS	DISCOUNT	VAT	NET INVOICED	COLLECTED	OUTSTANDING																		
01 Aug 2026	6	123,550.00	1,052.50	22,049.55	144,547.05	134,402.00	10,145.05																		
02 Aug 2026	2	324,200.00	0.00	58,356.00	382,556.00	382,556.00	0.00																		

Test Case ID	T-016																																																																													
Function	Audit Trail																																																																													
Input Data	Signed in as admin, open Administration then Audit Trail																																																																													
Expected Result	Every sign-in and every change is listed with its IP address																																																																													
Actual Result	Audit entries displayed, written by both the application and the database triggers																																																																													
Screenshot	Sunrise Dental Clinic Compact view Samanthi Rajapaksa Administrator Sign out Dashboard Appointments New Appointment Patients Billing Reports Dentists Treatments Administration Help Audit Trail Who did what, and when. Append-only — entries are never edited or deleted. Notification history Staff accounts System diagnostics Filter Username Action contains e.g. reception LOGIN, APPOINTMENT, INVOICE, ... Filter Clear Entries (405) <table><tr><th>WHEN</th><th>USER</th><th>ACTION</th><th>ENTITY</th><th>DETAILS</th><th>SOURCE</th><th>IP</th></tr><tr><td>11 Aug 21:35:06</td><td>admin</td><td>LOGIN_SUCCESS</td><td>USER admin</td><td>Signed in successfully</td><td>Application</td><td>0:0:0:0:0:0:1</td></tr><tr><td>11 Aug 21:34:19</td><td>reception</td><td>APPOINTMENT_CREATED</td><td>APPOINTMENT APT-2026-000404</td><td>Booked for 2026-09-01 18:00, status CONFIRMED</td><td>Database</td><td>-</td></tr><tr><td>11 Aug 21:34:19</td><td>reception</td><td>APPOINTMENT_CREATED</td><td>APPOINTMENT APT-2026-000403</td><td>Booked for 2026-09-01 10:00, status CONFIRMED</td><td>Database</td><td>-</td></tr><tr><td>11 Aug 21:34:19</td><td>reception</td><td>APPOINTMENT_CREATED</td><td>APPOINTMENT APT-2026-000402</td><td>Booked for 2026-09-01 15:00, status CONFIRMED</td><td>Database</td><td>-</td></tr><tr><td>11 Aug 21:34:19</td><td>reception</td><td>APPOINTMENT_CREATED</td><td>APPOINTMENT APT-2026-000401</td><td>Booked for 2026-08-31 15:00, status CONFIRMED</td><td>Database</td><td>-</td></tr><tr><td>11 Aug 21:34:19</td><td>reception</td><td>APPOINTMENT_CREATED</td><td>APPOINTMENT APT-2026-000400</td><td>Booked for 2026-08-31 15:00, status SCHEDULED</td><td>Database</td><td>-</td></tr><tr><td>11 Aug 21:34:19</td><td>reception</td><td>APPOINTMENT_CREATED</td><td>APPOINTMENT APT-2026-000399</td><td>Booked for 2026-08-31 08:30, status CONFIRMED</td><td>Database</td><td>-</td></tr><tr><td>11 Aug 21:34:19</td><td>reception</td><td>APPOINTMENT_CREATED</td><td>APPOINTMENT APT-2026-000398</td><td>Booked for 2026-08-31 14:00, status CONFIRMED</td><td>Database</td><td>-</td></tr><tr><td>11 Aug 21:34:19</td><td>reception</td><td>APPOINTMENT_CREATED</td><td>APPOINTMENT APT-2026-000397</td><td>Booked for 2026-08-31 13:30, status CONFIRMED</td><td>Database</td><td>-</td></tr><tr><td>11 Aug 21:34:19</td><td>reception</td><td>APPOINTMENT_CREATED</td><td>APPOINTMENT APT-2026-000396</td><td>Booked for 2026-08-30 14:00, status SCHEDULED</td><td>Database</td><td>-</td></tr></table>	WHEN	USER	ACTION	ENTITY	DETAILS	SOURCE	IP	11 Aug 21:35:06	admin	LOGIN_SUCCESS	USER admin	Signed in successfully	Application	0:0:0:0:0:0:1	11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000404	Booked for 2026-09-01 18:00, status CONFIRMED	Database	-	11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000403	Booked for 2026-09-01 10:00, status CONFIRMED	Database	-	11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000402	Booked for 2026-09-01 15:00, status CONFIRMED	Database	-	11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000401	Booked for 2026-08-31 15:00, status CONFIRMED	Database	-	11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000400	Booked for 2026-08-31 15:00, status SCHEDULED	Database	-	11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000399	Booked for 2026-08-31 08:30, status CONFIRMED	Database	-	11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000398	Booked for 2026-08-31 14:00, status CONFIRMED	Database	-	11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000397	Booked for 2026-08-31 13:30, status CONFIRMED	Database	-	11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000396	Booked for 2026-08-30 14:00, status SCHEDULED	Database	-
WHEN	USER	ACTION	ENTITY	DETAILS	SOURCE	IP																																																																								
11 Aug 21:35:06	admin	LOGIN_SUCCESS	USER admin	Signed in successfully	Application	0:0:0:0:0:0:1																																																																								
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000404	Booked for 2026-09-01 18:00, status CONFIRMED	Database	-																																																																								
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000403	Booked for 2026-09-01 10:00, status CONFIRMED	Database	-																																																																								
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000402	Booked for 2026-09-01 15:00, status CONFIRMED	Database	-																																																																								
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000401	Booked for 2026-08-31 15:00, status CONFIRMED	Database	-																																																																								
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000400	Booked for 2026-08-31 15:00, status SCHEDULED	Database	-																																																																								
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000399	Booked for 2026-08-31 08:30, status CONFIRMED	Database	-																																																																								
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000398	Booked for 2026-08-31 14:00, status CONFIRMED	Database	-																																																																								
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000397	Booked for 2026-08-31 13:30, status CONFIRMED	Database	-																																																																								
11 Aug 21:34:19	reception	APPOINTMENT_CREATED	APPOINTMENT APT-2026-000396	Booked for 2026-08-30 14:00, status SCHEDULED	Database	-																																																																								

Test Case ID	T-017
---------------------	-------

Function	Notification History
Input Data	Open Administration then Notifications
Expected Result	Every alert attempt is listed, successful or failed
Actual Result	Notification log displayed with the channel and delivery status
Screenshot	

Test Case ID	T-018
Function	Web Service Documentation
Input Data	Open /swagger-ui.html
Expected Result	The versioned REST contract is published and browsable
Actual Result	OpenAPI documentation displayed for every /api/v1 endpoint
Screenshot	

5. Test Automation and Results

The whole suite is automated and runs from a single command, `mvn verify`. Nothing needs to be installed first and no database needs to be started, because the integration tests use an in-memory H2 instance that Flyway migrates on the way up. The same command runs in continuous integration on every push, on both JDK 17 and JDK 21, and the workflow then packages both jars and boots the packaged server to prove it starts.

Measure	Result
Unit tests	226 - all passing, 0 failures, 0 errors, 0 skipped
Integration tests	97 - all passing, 0 failures, 0 errors, 0 skipped
Total	323 automated tests, all passing
Test classes	31 (26 unit, 5 integration)
Code coverage (JaCoCo)	73.7% lines, 76.4% methods, 96.6% classes on dental-server
Build	<code>mvn verify</code> green on JDK 17 and JDK 21

6. Defects Found and Fixed

A test report claiming that everything passed first time is not one anybody should believe. Three genuine defects were found after the suite was already green, all by the concurrency test. Two of them shared a single root cause worth stating plainly: a failure that surfaces at transaction commit, inside Spring's proxy, cannot be caught by a try/catch inside the same transactional method.

Defect	Symptom	Fix
Login bookkeeping race	Two requests authenticating as the same account in the same instant raced to update one versioned row. The loser's locking failure escaped as a raw HTTP 500 and its audit entry was lost.	The update was moved into its own transactional bean, so the listener can see the failure; the audit entry is now written unconditionally.
First sequence allocation	SELECT ... FOR UPDATE cannot lock a row that does not exist yet, so the first appointment number of a year could be claimed by several bookings at once and every loser's booking was abandoned.	A one-time retry inside a genuinely new transaction.
CI action reference	The security-scan job referenced an action tag without its v prefix, so the job would have failed on its first run.	Corrected, and every action reference was then verified against the GitHub API.

The lesson generalises. A sequential test proves that a rule is written; only a concurrent test proves that it holds. Both defects would have reached production and been very difficult to reproduce from a bug report, because they appear only when two people act in the same instant, which is precisely what a busy front desk creates every day.

7. Traceability

Requirement	Implementation	Verified by
1. User authentication	Spring Security, two filter chains, BCrypt	T-001, T-002, T-003, SecurityIT
2. Register new appointment	AppointmentService with the six-handler validation chain	T-004, T-005, T-006, AppointmentApiIT
3. Display appointment details	Search by number, name or telephone	T-007, T-008, AppointmentApiIT
4. Calculate and print bill	BillingService with four pricing strategies	T-009 to T-014, PricingStrategyTest, BillingApiIT
5. Help section	HelpService and the /help pages	ReportAndWebUiIT
6. Exit system	POST-only sign-out; desktop exit command	T-020, SecurityIT
Distributed with web services	Versioned /api/v1, separate client	T-017, T-018
Design patterns	Thirteen patterns, Task B section 2 onwards	PricingStrategyFactoryTest, BookingValidationChainTest
Proper database	9 tables, 5 views, 2 functions, 2 procedures, 4 triggers	DatabaseFeaturesIT, ClinicFunctionsTest
Validation	Four layers, Task B section 8	T-005, T-012, ValidationConstraintsTest
Reports	Five reports over database views	T-015, ReportAndWebUiIT

Task D

1. GitHub Repository

GitHub Repository Link: [https://github.com/\[your-github-username\]/sunrise-dental-clinic](https://github.com/[your-github-username]/sunrise-dental-clinic)

The repository is public, so it can be opened without a GitHub account. It contains the complete Maven project, the migration scripts, the test suite, both CI workflows, this documentation and the source of every diagram.

2. Version Control Techniques

The history was built deliberately rather than allowed to happen. Each commit is a complete, working step, so the repository shows how the application was constructed rather than appearing fully formed in a single commit. Any commit can be checked out and built.

Technique	How it was applied	Why
Conventional Commits	feat(pricing);, fix(security);, test; ci; docs;	The subject line says what changed and where, so the log is searchable and a changelog can be assembled from it.
Meaningful increments	One commit per architectural layer or feature	A reviewer can follow the reasoning; a single final-submission commit hides all of it.
Branching	main holds released work; develop integrates	A simplified Git Flow. A full feature-branch model would be ceremony for a single developer.
Semantic versioning	Annotated tags v1.0.0 and v1.0.1	The version number states whether an upgrade is safe.
Tag-driven release	Pushing a v*.*.* tag triggers the release	A deployed version always traces back to one immutable commit.
Ignore rules	Build output, IDE files, the local database and credentials are excluded	The repository is public, so patient data must never reach it.
Changelog	CHANGELOG.md in Keep a Changelog format	Explains why each release exists, which a commit log alone does not.

3. Workflows Deployed with the Repository

Continuous integration (ci.yml) runs on every push and every pull request. It builds on a matrix of JDK 17 and JDK 21 - the project targets 17 for portability, and the matrix proves it also runs on a newer runtime - then publishes the test and coverage

reports as artefacts, packages both executable jars, and boots the packaged server to confirm it starts and answers its health check. A separate job runs a dependency vulnerability scan.

Release (release.yml) is triggered only by a tag matching `v*.*.*`. It rebuilds from the tagged commit and publishes a GitHub Release carrying both jars and that version's changelog section, so a release can never be published from uncommitted work.

Together the two workflows mean that the state of the main branch is always known - it either has a green tick or it does not - and that producing a release is one command rather than a checklist somebody can get wrong.

4. Evaluation

One specific instance shows the approach working. When the concurrency defects described in Task C were found after `v1.0.0` had already been tagged, the fix was three focused commits and a `v1.0.1` tag. Because the history was granular and the messages conventional, it was possible to see exactly which commit had introduced the login bookkeeping and what it had been trying to achieve. That is the practical value of a disciplined history: the ability to answer "why is this like this?" months later.

References

- Beck, K. (2003) *Test-Driven Development: By Example*. Boston: Addison-Wesley. Available at: <https://www.oreilly.com/library/view/test-driven-development/0321146530/> (Accessed: 12 July 2026).
- Bloch, J. (2018) *Effective Java*. 3rd edn. Boston: Addison-Wesley. Available at: <https://www.oreilly.com/library/view/effective-java/9780134686097/> (Accessed: 14 July 2026).
- Driessen, V. (2010) *A successful Git branching model*. Available at: <https://nvie.com/posts/a-successful-git-branching-model/> (Accessed: 28 July 2026).
- Fowler, M. (2003) *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd edn. Boston: Addison-Wesley. Available at: <https://martinfowler.com/books/umlDistilled.html> (Accessed: 18 July 2026).
- Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley. Available at: <https://martinfowler.com/books/ea.html> (Accessed: 20 July 2026).
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley. Available at: <https://www.oreilly.com/library/view/design-patterns-elements/0201633612/> (Accessed: 10 July 2026).
- GitHub (2026) *GitHub Actions documentation*. Available at: <https://docs.github.com/en/actions> (Accessed: 02 August 2026).
- Martin, R.C. (2008) *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall. Available at: <https://www.oreilly.com/library/view/clean-code-a/9780136083238/> (Accessed: 16 July 2026).
- Object Management Group (2017) *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. Available at: <https://www.omg.org/spec/UML/2.5.1/> (Accessed: 20 July 2026).
- Oracle (2024) *Java Platform, Standard Edition 17 Documentation*. Available at: <https://docs.oracle.com/en/java/javase/17/> (Accessed: 15 July 2026).

- OWASP Foundation (2021) *OWASP Top Ten Web Application Security Risks*. Available at: <https://owasp.org/www-project-top-ten/> (Accessed: 25 July 2026).
- Preston-Werner, T. (2013) *Semantic Versioning 2.0.0*. Available at: <https://semver.org/> (Accessed: 28 July 2026).
- Red Gate Software (2024) *Flyway Documentation*. Available at: <https://documentation.red-gate.com/flyway> (Accessed: 18 July 2026).
- VMware (2025) *Spring Boot Reference Documentation, version 3.5*. Available at: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (Accessed: 10 July 2026).
- VMware (2025) *Spring Security Reference*. Available at: <https://docs.spring.io/spring-security/reference/> (Accessed: 22 July 2026).